

Final Project: Evaluating LLMs on Text-to-SQL downstream task

Overview

In this project, you will build on the concepts and tools from **sessions related Text-to-SQL Query Generation**. Your task is to evaluate the performance of a large language model (LLM) on a real-world **Text-to-SQL** dataset using one of two options (datasets) provided below.

Objective

You will reuse your solution from Exercises 8 and 9 (i.e., using LangChain or a similar approach to translate natural language into SQL) and apply it to a **new dataset**. The goal is to assess the effectiveness of your LLM-based system by measuring **execution accuracy** on selected queries.

Dataset Options

Choose **one** of the following datasets:

1. **Football Dataset**
 - Includes information about players, matches, and statistics.
 - Refer to the provided documentation for details on accessing the database and the test set.
2. **StatBotSwiss Dataset**
 - Contains data from the Swiss Federal Statistical Office.
 - Refer to the provided documentation for connection details and test queries.

Tasks

1. Dataset Selection

Choose one dataset and configure the access (Please adjust it if the method deprecated!)

```
from langchain_community.utilities import SQLDatabase
database_uri =
f"postgresql://{username}:{password}@{host}:{str(port)}/{database}"
sqldb = SQLDatabase.from_uri(database_uri)
sqldb._schema = "experiment"
```

The attached documents provide this information for each database:

Football DB and StatBotSwiss DB

Note: You need to be in the ZHAW network or use VPN to connect to the BDs.

2. Model Configuration

Use a language model of your choice (e.g., Llama2, GPT-4, Gemma) and configure it using one of the approaches from Session 2:

- LangChain's SQLDatabaseChain
- Chat-based prompt engineering with structured outputs
- LangGraph-based multi-step reasoning

3. Query Evaluation

Select **15-20 natural language questions** (50% easy and medium and 50% from hard and extra hard) from the dataset's test set. For each:

- Generate the corresponding SQL query using your model.
- Execute the query.
- Compare the output with the ground truth or expected answer.

4. Metric: Execution Accuracy

For each question, mark as **correct** if the executed result matches the expected output. Report your **overall accuracy**.

Suggested Techniques (Optional Enhancements)

You are encouraged to explore and include any of the following techniques to improve and analyze your model's performance:

- **In-context Learning**: Supply few-shot examples within the prompt.
- **Chain-of-Thought**: Use reasoning steps before generating SQL.
- **Self-Consistency**: Sample multiple outputs and select the most consistent result.
- **Self-Refine/Self-Reflection**: Let the model verify or critique its own SQL output.
- **Agentic Patterns**: Use tools or memory as part of a more autonomous agent.

Deliverables

Submit a **project report** (max 10 pages) that includes the following:

1. Dataset Choice & Rationale
2. System Architecture / Setup
3. Query Examples with Input, Generated SQL, and Result
4. Execution Accuracy Results
5. Analysis of Errors / Model Limitations
6. Optional Techniques Implemented
7. Conclusion and Future Work

Code Repository (via GitHub or .zip)

- Include all source files, scripts, and configuration
- A **Jupyter Notebook is preferred**, containing:
 - Code cells for each major step
 - Output cells showing query results and logs
 - Clear markdown explanations and inline comments

Evaluation Criteria:

Criterion	Points
Functionality and correctness	30
Execution accuracy	20
Code clarity and organization	15
Depth of analysis in report	20
Use of advanced techniques	10
Report structure and clarity	5
Total	100